VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

LOGIC DESIGN PROJECT

Project Report

FFT-BASED CONVOLUTION ACCELERATOR IN VERILOG

Class TN02 – Semester 251

Advisor(s): Quoc Cuong Pham
Student(s): Huu Thinh Nguyen 2313292
Chi Thanh Nguyen 2313078
Viet Hoang Nguyen 2311066

HO CHI MINH CITY, DECEMBER 2025

Contents

1 Theory and Background

1.1 Discrete Fourier Transform (DFT)

The Discrete Fourier Transform (DFT) is a fundamental tool in digital signal processing that maps a finite-length discrete-time signal from the time domain into the frequency domain. Given a discrete-time sequence $x(n)$ of length N, the DFT is defined as [1]:

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk}, \quad k = 0, 1, \ldots, N-1 \tag{1.1}$$

where the complex exponential term

$$W_N^{nk} = e^{-j\frac{2\pi nk}{N}} = \cos\left(\frac{2\pi nk}{N}\right) - j\sin\left(\frac{2\pi nk}{N}\right) \tag{1.2}$$

is referred to as the *twiddle factor*. In this formulation, $x(n)$ denotes the input sequence in the time domain, while $X(k)$ represents the discrete frequency-domain spectrum.

1.2 Fast Fourier Transform (FFT)

Given a discrete-time sequence $x(n)$ of length N, the N-point Discrete Fourier Transform (DFT) is defined as

$$X(k) = \sum_{n=0}^{N-1} x(n)\, W_N^{nk}, \quad k = 0, 1, \ldots, N-1, \tag{1.3}$$

where the twiddle factor is

$$W_N^{nk} \triangleq e^{-j\frac{2\pi nk}{N}}. \tag{1.4}$$

Equations(1.3)–(1.4) describe the frequency-domain representation of a discrete-time signal and form the mathematical basis of FFT algorithms [1].

Direct computation of(1.3) requires $\mathcal{O}(N^2)$ operations. The Fast Fourier Transform (FFT) reduces this cost by recursively decomposing the DFT into smaller transforms using the periodicity and symmetry of W_N [1].

1.2.1 Radix-2 DIT Decomposition.

In the radix-2 Decimation-in-Time (DIT) FFT, the input sequence is split into even- and odd-indexed samples:

$$x_e(r) = x(2r), \qquad x_o(r) = x(2r+1), \tag{1.5}$$

leading to

$$X(k) = E(k) + W_N^k O(k), \tag{1.6}$$

$$X\left(k + \frac{N}{2}\right) = E(k) - W_N^k O(k), \tag{1.7}$$

$$E(k) \triangleq \sum_{r=0}^{\frac{N}{2}-1} x_e(r)\, W_{N/2}^{rk}, \qquad k = 0, 1, \ldots, \frac{N}{2} - 1, \tag{1.8}$$

$$O(k) \triangleq \sum_{r=0}^{\frac{N}{2}-1} x_o(r)\, W_{N/2}^{rk}, \qquad k = 0, 1, \ldots, \frac{N}{2} - 1, \tag{1.9}$$

where $E(k)$ and $O(k)$ are $(N/2)$-point DFTs of the even and odd subsequences, respectively [1].

1.2.2 Radix-2 DIF Decomposition.

In the radix-2 Decimation-in-Frequency (DIF) FFT, the decomposition is applied in the frequency domain. The first butterfly stage computes

$$u(r) = x(r) + x\left(r + \frac{N}{2}\right), \tag{1.10}$$

$$v(r) = \left[x(r) - x\left(r + \frac{N}{2}\right)\right] W_N^r, \tag{1.11}$$

which are then processed by two independent $(N/2)$-point DFTs to obtain the even- and odd-indexed frequency components [1].

1.2.3 Computational Significance.

Both DIT and DIF FFT algorithms reduce the computational complexity of the DFT to

$$\mathcal{O}(N \log_2 N), \tag{1.12}$$

by organizing the computation into $\log_2 N$ stages of radix-2 butterfly operations, each exploiting the structure of the twiddle factors [1].

1.3 Convolution Computation Using FFT

In digital signal processing, the linear convolution between two finite-length discrete-time sequences $x(n)$ and $h(n)$ is defined as

$$y(n) = (x * h)(n) = \sum_{m=0}^{L_x-1} x(m)\, h(n - m), \tag{1.13}$$

where L_x and L_h denote the lengths of the input sequence $x(n)$ and the impulse response $h(n)$, respectively. The resulting output sequence $y(n)$ has a length of

$$L_y = L_x + L_h - 1.$$

A direct evaluation of (1.13) requires $\mathcal{O}(L_x L_h)$ multiplications, which becomes computationally expensive for long sequences.

A fundamental property of the Discrete Fourier Transform, known as the *Convolution Theorem*, states that linear convolution in the time domain corresponds to pointwise multiplication in the frequency domain. Specifically, if $x(n)$ and $h(n)$ are zero-padded to a common length $N \geq L_x + L_h - 1$, then

$$Y(k) = X(k)\, H(k), \tag{1.14}$$

where

$$X(k) = \mathrm{FFT}\{x(n)\}, \qquad H(k) = \mathrm{FFT}\{h(n)\}.$$

The linear convolution result can then be obtained by applying the inverse FFT:

$$y(n) = \mathrm{IFFT}\{X(k)\, H(k)\}\,. \tag{1.15}$$

In the proposed hardware architecture, the forward FFT is implemented using the radix-2 DIF algorithm, while the inverse FFT is realized using the radix-2 DIT algorithm.

1.3.1 FFT-Based Convolution Flow.

Based on (1.14) and (1.15), linear convolution can be efficiently implemented using the following steps:

1. Zero-pad $x(n)$ and $h(n)$ to a common length N.
2. Compute $X(k) = \mathrm{FFT}\{x(n)\}$ and $H(k) = \mathrm{FFT}\{h(n)\}$.
3. Perform pointwise complex multiplication: $Y(k) = X(k)H(k)$.
4. Apply the inverse FFT to obtain $y(n) = \mathrm{IFFT}\{Y(k)\}$.

1.3.2 Computational Complexity Comparison.

For sequences of length approximately N, direct time-domain convolution requires $\mathcal{O}(N^2)$ operations. In contrast, FFT-based convolution requires:

- Two FFT operations: $2\,\mathcal{O}(N \log_2 N)$,
- One inverse FFT: $\mathcal{O}(N \log_2 N)$,
- One pointwise complex multiplication: $\mathcal{O}(N)$.

Therefore, the overall computational complexity is

$$\mathcal{O}(N \log_2 N), \tag{1.16}$$

which is significantly lower than the direct convolution approach for large N.

1.3.3 Conclusion.

Due to its significantly reduced computational complexity and highly regular computational structure, FFT-based convolution is well suited for hardware implementation. The decomposition into FFT blocks, pointwise complex multipliers, and inverse FFT enables efficient pipelining, parallelism, and memory reuse. For these reasons, the proposed design adopts FFT-based convolution as the core computational strategy.

2 Specification

2.1 Algorithm Overview

The convolution of two signals, $x[n]$ and $h[n]$, is computationally intensive in the time domain. To accelerate this process, we utilize the Convolution Theorem, which states that convolution in the time domain corresponds to point-wise multiplication in the frequency domain.

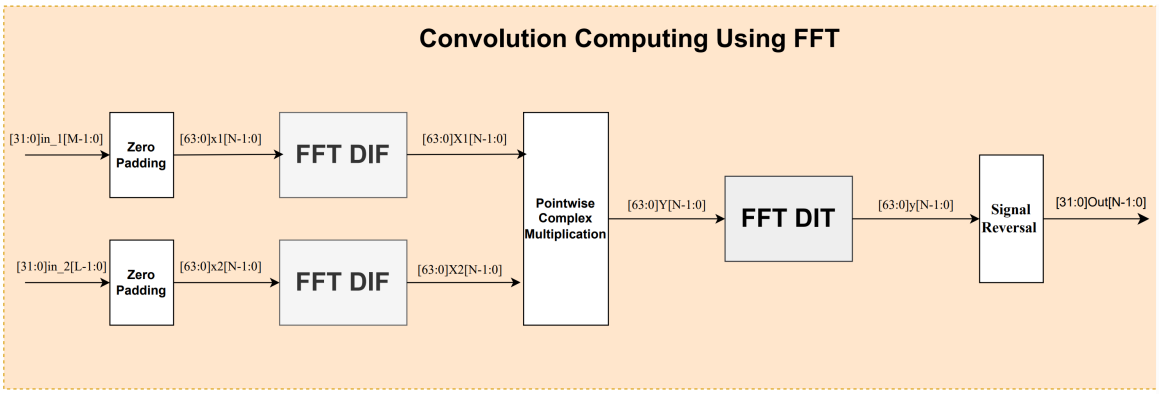

Figure 2.1: Convolution Compute using FFT Accelerator

2.2 Proposed Optimizations Architecture

To achieve an area-efficient design, we propose a resource-sharing strategy that exploits the arithmetic similarities between the processing stages.

Resource Sharing Optimization: An analysis of the computational flow reveals that the Decimation-in-Frequency (DIF) FFT, the Decimation-in-Time (DIT) IFFT, and the point-wise multiplication steps all rely heavily on complex multiplication and addition/subtraction. Rather than instantiating separate hardware modules for each stage, we design a unified **Arithmetic Logic Unit (ALU)** capable of configuring itself as a Butterfly unit or a complex multiplier. This approach significantly reduces the silicon area, albeit at the cost of increased complexity in the Control Unit state machine.

DIF-FFT and DIT-IFFT Pairing: A critical optimization in our design is the specific selection of FFT variants. We implement the forward transform using the **Radix-2 DIF** algorithm and the inverse transform using the **Radix-2 DIT** algorithm.

- The **DIF-FFT** produces output data in *bit-reversed* order.
- The **DIT-IFFT** expects input data in *bit-reversed* order to produce natural-order output.

By performing the intermediate point-wise multiplication directly on the bit-reversed data, we seamlessly connect the FFT output to the IFFT input. This eliminates the need for explicit data reordering buffers or logic, thereby reducing latency and memory addressing overhead.

2.3 Hardware Architecture

Based on the proposed architecture, the hardware implementation consists of the following key components:

- **RAM (Dual-port SRAM):** Provides on-chip storage for input sequences, intermediate spectral data ($X[k], Y[k]$), and the final output. It supports simultaneous read/write operations to maximize throughput.
- **Twiddle Factor ROM:** A lookup table storing the pre-computed trigonometric coefficients (W_N^k) required for the butterfly operations.
- **ALU (Reconfigurable Datapath):** The core computational engine. It executes complex additions and multiplications, dynamically switching roles between FFT Butterfly processing, spectral multiplication, and IFFT Butterfly processing.
- **Control Unit:** A Finite State Machine (FSM) that orchestrates the entire operation. It manages the complex addressing modes for bit-reversal/natural order access and controls the ALU's operating mode across the three processing stages.

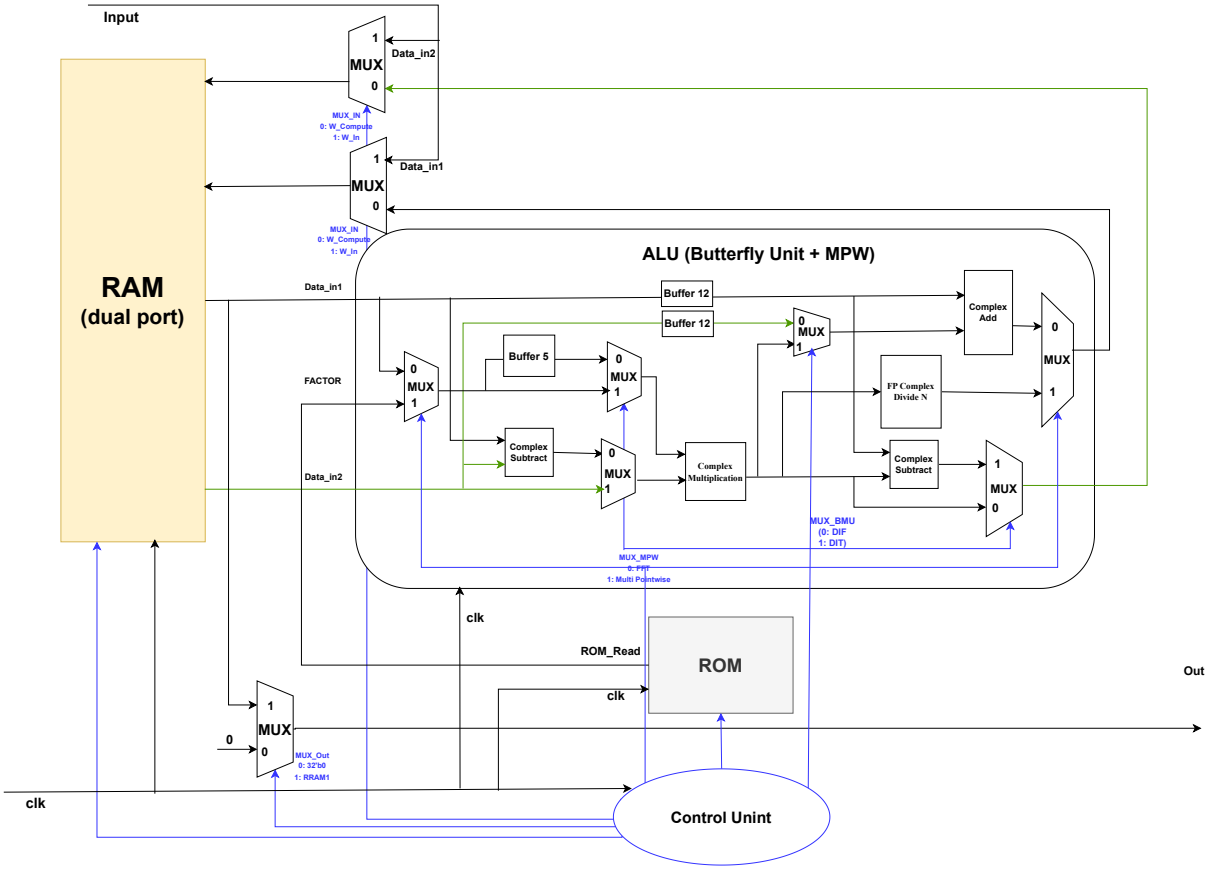

Figure 2.2: Architecture Convolution using FFT Accelerator

3 Implementation

3.1 Implementation of the RAM

3.1.1 Interface

The module operates on a single clock signal (`clk`). The interface definition is detailed in Table 3.1. The figure 3.1 illustrates the block schematic of the True Dual-Port RAM.

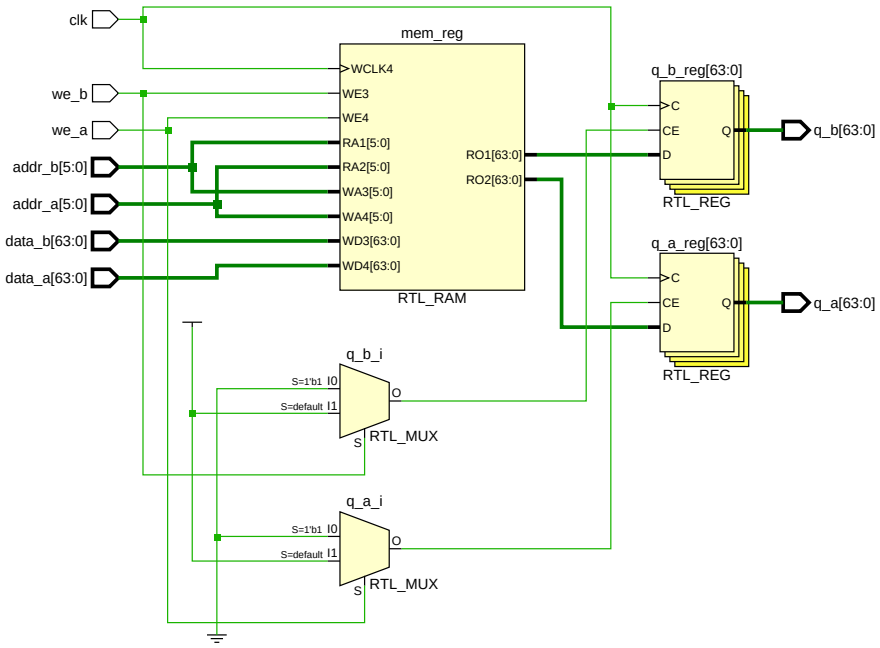

Figure 3.1: True Dual-Port RAM Block Schematic

3.1.2 Functional Behavior

The RAM operates under the following conditions:

- **Synchronous Operation:** All memory transactions occur on the rising edge of `clk`.
- **Dual Access:** Port A and Port B operate independently and can be active simultaneously.
- **Read/Write Modes:**
  - **Write Mode (`we = 1`):** The input data is written to the specified address.
  - **Read Mode (`we = 0`):** The data at the specified address is registered into the output `q`.

Table 3.1: RAM Interface Signals

Port Name	Direction	Width	Description
clk	Input	1	System clock (rising edge triggered).
Port A			
data_a	Input	DATA_WIDTH	Data to be written to memory.
addr_a	Input	ADDR_WIDTH	Address for read/write operations.
we_a	Input	1	Write Enable (High = Write, Low = Read).
q_a	Output	DATA_WIDTH	Data read from memory.
Port B			
data_b	Input	DATA_WIDTH	Data to be written to memory.
addr_b	Input	ADDR_WIDTH	Address for read/write operations.
we_b	Input	1	Write Enable (High = Write, Low = Read).
q_b	Output	DATA_WIDTH	Data read from memory.

However, simultaneous read and write operations to the same address from different ports will lead to undefined behavior.

3.1.3 Storage Structure

The memory core is modeled abstractly as a two-dimensional array of registers. This structure is sized dynamically based on the DATA_WIDTH and MEM_DEPTH parameters. The model is specifically designed to allow synthesis tools to infer dedicated hardware Block RAM (BRAM) resources on FPGA targets, ensuring optimal area usage and timing performance compared to distributed logic storage.

3.1.4 Parallel Control Logic

To achieve True Dual-Port functionality, the design employs two distinct, parallel control processes triggered by the rising edge of the clock. Each process manages the operations of a single port (A or B) independently. Since both processes share access to the same underlying storage array, the component supports simultaneous Read/Read, Read/Write, and Write/Write operations across the two ports.

3.1.5 Read/Write Arbitration Logic

The internal logic for each port enforces a mutually exclusive operation mode based on the Write Enable (`we`) signal:

1. **Write Priority:** When the write enable signal is active, the system prioritizes the storage operation. The data present on the input bus is written directly into the memory array at the specified address.
2. **Output Behavior during Write:** The implementation utilizes a "Read-Disabled" (or No-Change) architecture during write cycles. This means that while data is being written to memory, the output data port is *not* updated with the new input nor the old memory content. Instead, it retains the value held from the previous clock cycle.
3. **Read Operation:** Only when the write enable signal is inactive does the system retrieve data from the memory array and update the output register.

This logic structure minimizes the hardware overhead by avoiding complex "Read-First" or "Write-First" arbitration circuitry, making it highly efficient for standard buffer and storage applications.

3.2 Twiddle Factor ROM Design and Implementation

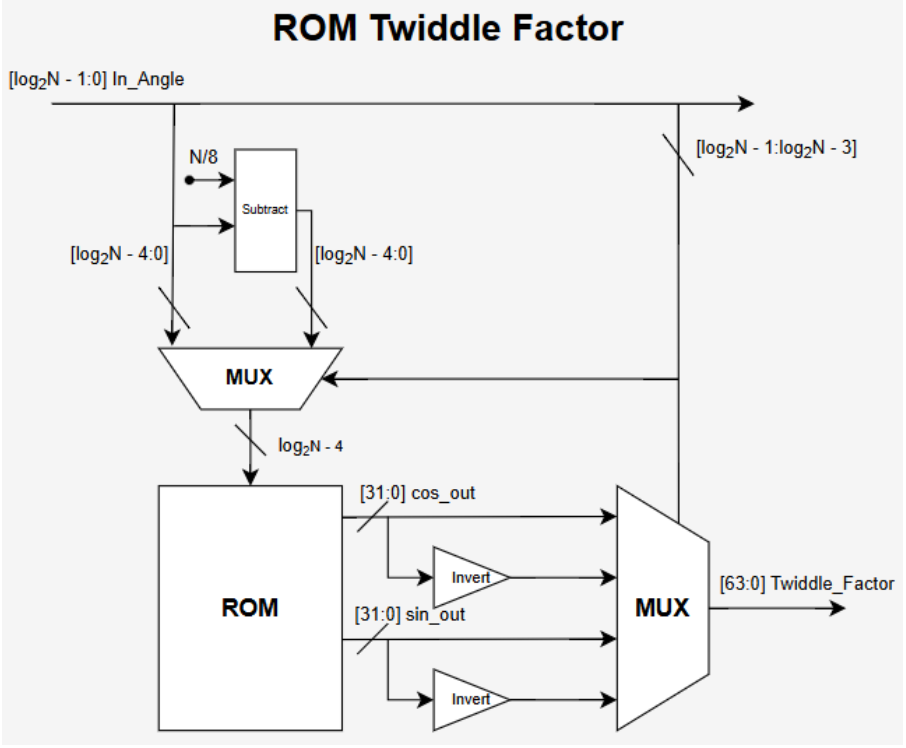

Figure 3.2: The architecture of ROM twiddle factor

The twiddle factor ROM is designed to efficiently generate complex exponential coefficients required by FFT butterfly operations while minimizing memory usage. Instead of storing the full set of N twiddle factors, the proposed architecture stores sine and cosine values only for the first octant of the unit circle, corresponding to the angular range $[0, \pi/4)$. All remaining twiddle factors are reconstructed using symmetry properties of trigonometric functions.

The twiddle factor is defined as

$$W_N^k = e^{-j\frac{2\pi k}{N}} = \cos(\beta) - j\sin(\beta), \quad \beta = \frac{2\pi k}{N}. \tag{3.1}$$

By exploiting symmetry, only $N/8$ base values are stored in ROM, leading to an eightfold reduction in memory size. The complete twiddle factor reconstruction is achieved using address mirroring, sine–cosine swapping, and sign inversion.

3.2.1 Symmetrical Mapping of Sine and Cosine

Table 3.2 summarizes the symmetry rules used to reconstruct sine and cosine values over the full range $[0, 2\pi)$ from the first-octant base interval.

Table 3.2: Symmetrical mapping of twiddle factors

Angle range β	$\sin(\beta)$	$\cos(\beta)$
$0 \sim \pi/4$	$-\sin(\beta)$	$\cos(\beta)$
$\pi/4 \sim \pi/2$	$-\cos(\frac{\pi}{2} - \beta)$	$\sin(\frac{\pi}{2} - \beta)$
$\pi/2 \sim 3\pi/4$	$-\cos(\beta - \frac{\pi}{2})$	$-\sin(\beta - \frac{\pi}{2})$
$3\pi/4 \sim \pi$	$-\sin(\pi - \beta)$	$-\cos(\pi - \beta)$
$\pi \sim 5\pi/4$	$\sin(\beta - \pi)$	$-\cos(\beta - \pi)$
$5\pi/4 \sim 3\pi/2$	$\cos(\frac{3\pi}{2} - \beta)$	$-\sin(\frac{3\pi}{2} - \beta)$
$3\pi/2 \sim 7\pi/4$	$\cos(\beta - \frac{3\pi}{2})$	$\sin(\beta - \frac{3\pi}{2})$
$7\pi/4 \sim 2\pi$	$\sin(2\pi - \beta)$	$\cos(2\pi - \beta)$

The mapping rules in Table 3.2 form the theoretical basis of the ROM reconstruction logic and are directly reflected in the Verilog implementation.

3.2.2 Module-Level Implementation Overview

Fig. 3.2 illustrates the structural organization of the twiddle factor generator. The design is composed of several lightweight combinational modules, each responsible for a specific reconstruction step:

- **`mux_addr` (Address Mapping Unit)** This module maps the intra-octant offset to a valid ROM address. For odd octants, address mirroring is applied using

$$\texttt{rom\_addr} = (N/8) - \texttt{off},$$

  ensuring that all accesses fall within the first-octant ROM range. This block is purely combinational and introduces no clock latency.

- **`rom_twiddle` (Compact Sine/Cosine ROM)** This module stores precomputed sine and cosine values for angles in $[0, \pi/4)$. The ROM is implemented as a combinational lookup table initialized from external data files. Once the address is stable, the corresponding sine and cosine values propagate directly to the output.

- **`Inv_FP` (Sign Inversion Units)** These modules generate negated sine and cosine values by flipping the IEEE-754 sign bit. This method avoids arithmetic subtraction and significantly reduces hardware complexity. The sign inversion logic is fully combinational.

- **`mux_output` (Symmetry Reconstruction Unit)** Based on the octant index, this module selects between sine and cosine base values, applies optional sign inversion, and performs sine–cosine swapping. The implemented selection logic directly follows the symmetry rules listed in Table 3.2.

- **`rom_twiddle_top` (Top-Level Twiddle Generator)** This top-level module integrates all submodules and produces the final complex twiddle factor. Boundary cases, where the offset equals zero, are handled using predefined constants (e.g., ± 1, 0, $\pm\sqrt{2}/2$) to avoid unnecessary ROM access and improve numerical robustness.

Although all reconstruction logic is combinational, the generated twiddle factor is synchronized at the system level. This ensures deterministic timing behavior and seamless integration with the pipelined FFT butterfly units.

Overall, the proposed ROM design achieves substantial memory reduction while maintaining exact trigonometric correctness, making it well suited for high-performance FFT hardware implementations.

3.3 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) serves as the computational core of the system, designed to execute floating-point (FP) arithmetic operations with high throughput. The architecture is fully pipelined to optimize processing speed while maintaining a compact hardware footprint. The ALU design addresses three distinct operational scenarios:

1. Radix-2 Decimation-in-Frequency (DIF) Butterfly operation.
2. Radix-2 Decimation-in-Time (DIT) Butterfly operation.
3. Point-wise Multiplication (MPW) for spectral convolution.

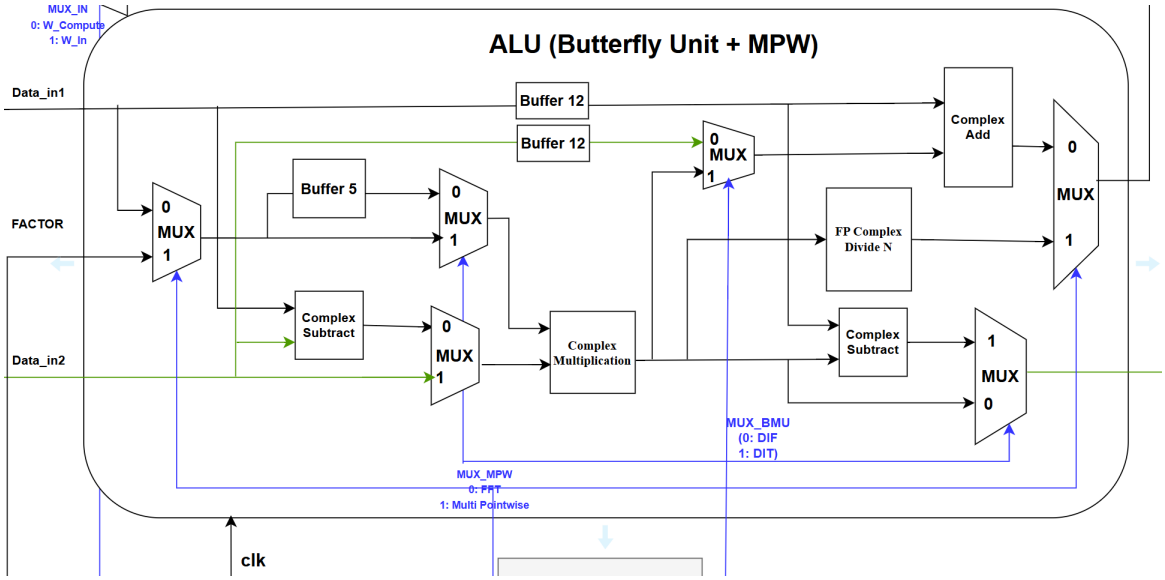

Figure 3.3: Block Diagram of Arithmetic Logic Unit (ALU)

Data Path Control and Temporal Alignment

To support the multi-mode functionality, the datapath employs a network of 64-bit multiplexers (MUX) to dynamically route operands based on the selected operation mode (controlled by signals `MUX_BMU` and `MUX_MPW`). A critical challenge in pipelined complex arithmetic is ensuring that operands arrive at the arithmetic units (Adders/Multipliers) at the exact same clock cycle.

To address this, we integrated specific delay buffers (as illustrated in the block diagram) to achieve **temporal alignment**. These buffers compensate for the latency differences between parallel paths, ensuring the correctness of the algorithm and the integrity of the pipeline stages.

3.3.1 Addition Floating Point Unit

The proposed Floating-Point Adder architecture is compliant with the IEEE 754 single-precision standard and is implemented using a 5-stage pipeline to maximize throughput. The datapath is decomposed into discrete functional blocks, utilizing Sign-Magnitude representation for efficient internal processing. The operation of each pipeline stage is described as follows:

- **Stage 1: Exponent Comparison (`AFP_Compare`):** This stage compares the exponents of the two operands (E_A, E_B). The absolute difference $\delta = |E_A - E_B|$ is calculated to

determine the alignment shift amount. Simultaneously, the larger operand is identified to guide the datapath routing, and the hidden bit is appended to form 24-bit significands.

- **Stage 2: Mantissa Alignment (`AFP_Shift`):** To enable arithmetic operations, the significand of the smaller number is right-shifted by δ positions. This is implemented using a hardware *Barrel Shifter*, ensuring that the binary points of both operands are aligned within a single clock cycle.
- **Stage 3: Arithmetic Operation (`AFP_Add`):** The core addition or subtraction is performed based on the effective operation sign. The ALU computes $M_{sum} = M_{large} \pm M_{shifted}$. The result is then converted back to Sign-Magnitude format (taking the absolute value if the result is negative) to prepare for normalization.
- **Stage 4: Normalization Detection (`AFP_Normal`):** To handle potential leading zeros resulting from subtraction, this stage employs a *Priority Encoder* (Leading One Detector). It scans the arithmetic result to determine the necessary left-shift amount (L_{shift}) required to restore the significand to the normalized $1.xxxx$ format.
- **Stage 5: Normalization and Rounding (`AFP_Rounding`):** The final stage executes the normalization shift ($M_{final} = M_{sum} \ll L_{shift}$) and adjusts the exponent accordingly ($E_{final} = E_{large} - L_{shift}$). Finally, Round-to-Nearest logic is applied to the guard bits, and the data is packed into the 32-bit IEEE 754 output format.

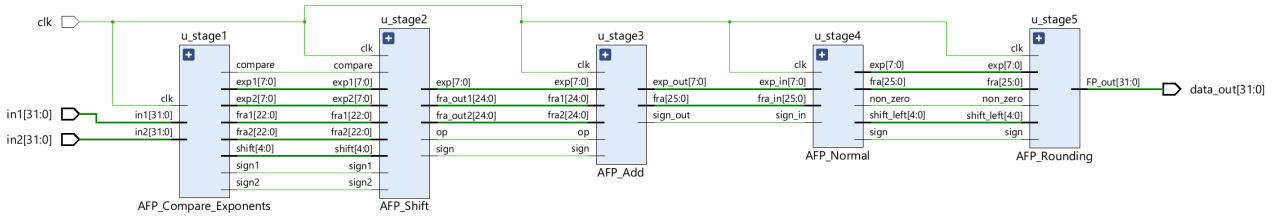

Figure 3.4: 5-Stage Pipelined Architecture of the Floating Point Adder

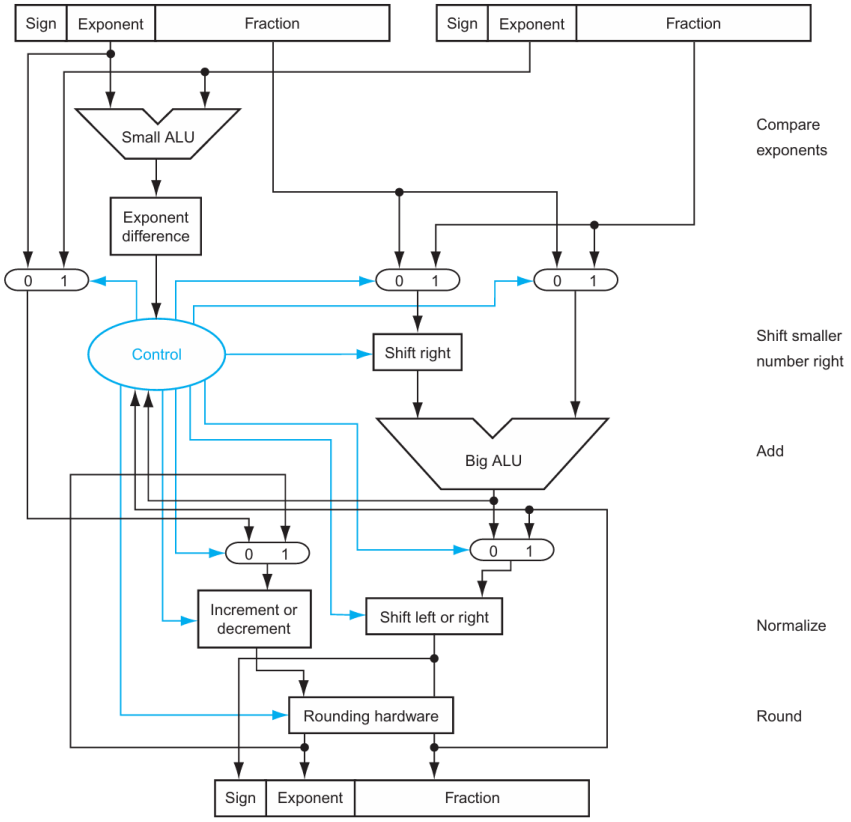

Figure 3.5: Architecture of the Floating Point Adder

3.3.2 Multiplication Floating Point Unit

To maintain high throughput and strictly adhere to the timing constraint of $T_{clk} \leq 8$ns, the multiplier architecture avoids a single-cycle 24×24-bit operation. Instead, we implement a multi-stage pipeline based on a **4-bit Nibble Decomposition Strategy**. This approach distributes the computational load across 7 pipeline stages (6 for accumulation and 1 for normalization).

Partial Product Accumulation (Stages 1-6): The core multiplication logic decomposes the 24-bit operand B into six 4-bit nibbles. In each stage i ($i = 0..5$), a 24×4-bit multiplication is performed, and the result is accumulated with the shifted partial sum from the previous stage according to the recursive formula:

$$P_i = (P_{i-1} \gg 4) + (M_A \times M_B[4i + 3 : 4i]) \tag{3.2}$$

This design ensures that the combinational depth of each stage remains shallow, effectively fitting within the required clock period.

Normalization and Final Packaging (Final Stage): The output of the accumulation pipeline is a 48-bit raw mantissa. The final stage executes post-processing operations to comply

with the IEEE 754 standard:

- **Normalization:** The leading non-zero bit is detected. If necessary, the mantissa is shifted, and the exponent is adjusted to ensure the format $1.xxxx$.
- **Rounding:** Round-to-Nearest logic is applied to truncate the result to 23 fraction bits, after which the Sign, Exponent, and Mantissa are packed into the final 32-bit register.

3.3.3 Complex Adder Unit

The Complex Adder Unit serves as a structural wrapper designed to perform addition on complex numbers according to the formula $(R_1 + jI_1) + (R_2 + jI_2) = (R_1 + R_2) + j(I_1 + I_2)$. To maximize throughput, the architecture exploits the inherent independence of the real and imaginary components.

Data Organization and Routing: The system utilizes a 64-bit data bus to represent a complex number. The input signals are split into two distinct 32-bit channels:

- **Imaginary Part (I):** Occupies the upper 32 bits ($[63:32]$).
- **Real Part (R):** Occupies the lower 32 bits ($[31:0]$).

Parallel Processing: The hardware implementation instantiates two parallel Floating-Point Adder cores (as described in the previous section). These cores share the same system clock and control signals but process data independently. Consequently, the Real and Imaginary sums are computed simultaneously, ensuring that the latency of the Complex Adder remains identical to that of a single FP Adder (5 clock cycles).

Subtraction Operation: same as addition, but the second operand is negated bit 32 of the floating point before feeding into the FP Adders.

3.3.4 Complex Multiplier Unit

Architectural Decision: To ensure pipeline regularity and simplify timing closure, the design adopts the **Standard Four-Multiplication Architecture** over the Gauss algorithm. This choice avoids the structural complexity of pre/post-adders inherent in the Gauss method, prioritizing a uniform datapath flow.

Pipelined Implementation and Timing Guarantee: The unit performs complex multiplication $Z = (AC - BD) + j(AD + BC)$ by decomposing 64-bit inputs into 32-bit floating-point components. The architecture guarantees a fixed total latency of **12 clock cycles**, structured

as follows:

- **Phase 1 (Cycles 1–7):** Four pipelined Floating-Point Multipliers operate in parallel to generate the partial products (AC, BD, AD, BC).
- **Phase 2 (Cycles 8–12):** The partial products are immediately routed to parallel Floating-Point Adders/Subtractors to compute the final Real ($AC - BD$) and Imaginary ($AD + BC$) parts.

This strict **12-cycle deterministic latency** ensures synchronization with the global control unit, preventing data hazards in the FFT processing stages.

3.3.5 Divider by N Unit

The normalization step in the IFFT algorithm requires dividing the output by the FFT length N. Since the system parameter N is constrained to be a power of 2 ($N = 2^k$), implementing a full floating-point divider is computationally redundant. Instead, we employ an optimized **Exponent Subtraction** technique.

Operation Principle: Dividing a floating-point number X by 2^k is equivalent to subtracting k from its exponent component.

$$E_{out} = E_{in} - \log_2(N) \tag{3.3}$$

The Sign bit and Mantissa field are passed through to the output unchanged, significantly reducing logic resources compared to a standard divider.

Exception Handling (Zero & Underflow): To ensure numerical correctness, the unit includes logic to handle boundary cases. The output is forced to the standard IEEE 754 Zero format ($32'h00000000$) if:

- The input value is Zero.
- The subtraction results in a negative exponent ($E_{in} < \log_2(N)$), indicating an underflow where the result is too small to be represented as a normalized number.

3.3.6 System Integration and Latency Characterization

Following the implementation and verification of individual functional modules, the final design phase focused on top-level integration and global routing. A critical objective was to establish a **Pipeline Synchronization Strategy**. By precisely characterizing the latency of each sub-

module, we inserted appropriate delay buffers at the input stages to align data streams, ensuring that operands arrive at the arithmetic cores at the exact required clock cycle.

Based on the synthesized datapath, the total processing latency for each operational mode is determined as follows:

- **FFT (DIF) and IFFT (DIT) Modes: 17 Clock Cycles.**
  The Butterfly operation inherently requires the serial execution of a Complex Addition/Subtraction and a Complex Multiplication.

$$\text{Latency}_{Butterfly} = \text{Lat}_{ComplexAdd}(5) + \text{Lat}_{ComplexMul}(12) = 17 \text{ cycles}$$

- **Multi-Point Wise (MPW) Mode: 13 Clock Cycles.**
  This operation consists of the spectral multiplication followed by the normalization (Divide by N) stage.

$$\text{Latency}_{MPW} = \text{Lat}_{ComplexMul}(12) + \text{Lat}_{DivN}(1) = 13 \text{ cycles}$$

Conclusion: The global architecture successfully manages inter-stage latencies. By strictly controlling the delay constraints for DIF, DIT, and MPW modes, the system ensures a seamless, stall-free pipeline flow, guaranteeing data integrity throughout the convolution process.

3.4 Control Unit Design

The Control Unit (CU) serves as the central orchestrator, managing the system's operation through a deterministic Finite State Machine (FSM). It works in tandem with an Address Generation Unit (AGU) to supply precise memory pointers and strictly manages the pipeline latency to ensure data integrity.

3.4.1 Finite State Machine (FSM) Architecture

The system operation is modeled as a sequential state transition flow. Upon activation, the FSM progresses through the following lifecycle:

$$\texttt{S\_IDLE} \xrightarrow{\text{Start}} \texttt{S\_LOAD} \rightarrow \underbrace{\texttt{S\_FFTX} \rightarrow \texttt{S\_FFTH}}_{\text{Forward Transforms}} \rightarrow \texttt{S\_MPW} \rightarrow \underbrace{\texttt{S\_IFFT}}_{\text{Inverse Transform}} \rightarrow \texttt{S\_OUT} \rightarrow \texttt{S\_DONE}$$

This linear progression ensures that the memory is initialized, the convolution is computed in the frequency domain, and the results are reconstructed before the system signals completion.

3.4.2 Address Generation Unit (AGU)

To minimize hardware area, the AGU shares arithmetic logic between the forward and inverse transforms.

1. Decimation-in-Frequency (DIF) Strategy: For the Forward FFT stages (`S_FFTX`, `S_FFTH`), the AGU calculates addresses based on the current `state` (representing the stage s) and `step` (butterfly index k). The addressing formulas are defined as:

$$\text{Distance} = \frac{N}{2^{\text{state}+1}} \tag{3.4}$$

$$\text{Addr}_{\text{RAM1}} = (\text{step}\%\text{Distance}) + \left(\frac{\text{step}}{\text{Distance}}\right) \times \text{Distance} \times 2 \tag{3.5}$$

$$\text{Addr}_{\text{RAM2}} = \text{Addr}_{\text{RAM1}} + \text{Distance} \tag{3.6}$$

$$\text{Addr}_{\text{Twiddle}} = (\text{step}\%\text{Distance}) \times \frac{N}{2 \times \text{Distance}} \tag{3.7}$$

2. Decimation-in-Time (DIT) Strategy: For the Inverse FFT stage (`S_IFFT`), the addressing logic utilizes the same hardware structure as the DIF strategy to maximize resource reuse. The key distinction lies in the traversal direction: while DIF increments the stage index from 0 to $\log_2 N - 1$, the DIT process operates in **reverse order**, decrementing the `state` from $\log_2 N - 1$ down to 0. This effectively inverts the butterfly hierarchy required for the inverse transform.

3.4.3 Latency Calculation and Pipeline Scheduling

To manage the pipeline depth effectively, the FSM organizes processing into discrete **computational batches**. The total system latency is deterministic and derived as follows:

1. FFT/IFFT Processing (Batch-based Execution): Each transform stage is divided into batches of 16 Butterfly operations. A single batch consists of a *Pipeline Fill* phase (17 cycles) and a *Pipeline Drain* phase (16 cycles), totaling 33 cycles.

- **Cycles per Stage:** With $N/32$ batches required per stage, the time to complete one stage is:

$$T_{\text{stage}} = \frac{N}{32} \times 33 \quad [\text{cycles}] \tag{3.8}$$

- **Total Transform Time:** For $\log_2 N$ stages, the latency for one full FFT/IFFT is:

$$T_{\text{transform}} = \frac{33N}{32} \log_2 N \quad [\text{cycles}] \tag{3.9}$$

2. Spectral Multiplication (MPW) Latency: The MPW stage processes data in blocks of 8. Each block incurs a latency of 21 cycles (13 Read + 8 Write).

$$T_{\text{MPW}} = \frac{N}{8} \times 21 = \frac{21N}{8} \quad [\text{cycles}] \tag{3.10}$$

3. Total System Latency: The total execution time from `start` to `done` sums the latencies of two Forward FFTs, one MPW, and one Inverse FFT:

$$T_{\text{Total}} \approx 2 \times T_{\text{transform}} + T_{\text{MPW}} + T_{\text{transform}} \tag{3.11}$$

Substituting the generalized formulas:

$$T_{\text{Total}} \approx 3 \times \left(\frac{33N}{32} \log_2 N \right) + \frac{21N}{8} \tag{3.12}$$

This formula demonstrates that the system performance scales linearly with $N \log N$, constrained primarily by the pipeline depth of the arithmetic units.

4 Results Synthesis and Implementation

The design was synthesized and implemented targeting the **Digilent Arty Z7-20** evaluation platform (Device: `xc7z020-clg400-1`) using the Xilinx Vivado Design Suite. The implementation configuration was optimized to leverage the specific resources of the Zynq-7000 SoC architecture. The following subsections detail the experimental results regarding hardware complexity, timing performance, and power consumption.

4.1 Hardware Complexity and Resource Utilization

The post-implementation resource usage is summarized in Table 4.1. The design fits comfortably within the target Zynq-7000 device constraints.

Analysis:

- **High Efficiency:** The design occupies less than 10% of the available Logic resources (LUTs). This low footprint validates the architectural decision to time-multiplex the ALU for FFT, MPW, and IFFT operations, effectively saving silicon area.
- **Memory Optimization:** Only 1.43% of Block RAM is utilized. The design relies primarily on distributed RAM (LUT as Memory: 197 units) for small buffers, preventing BRAM exhaustion and leaving resources available for other system components.

Table 4.1: Resource Utilization Summary (Device: xc7z020)

Resource Type	Used	Utilization (%)	Note
Slice LUTs	5,197	9.77%	Logic & Memory
Slice Registers	3,161	2.97%	Pipeline FFs
Slices	1,542	11.59%	-
Block RAM Tile	2	1.43%	FIFO & Buffers
Bonded IOB	3	2.40%	I/O Ports
BUFGCTRL	3	9.38%	Global Clocking

Name	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	Bonded IOB (125)	BUFGCTRL (32)
N Convolution_FTT	9.77%	2.97%	11.59%	9.40%	1.13%	1.43%	2.40%	9.38%
dut (ConvFTT)	9.76%	2.93%	11.59%	9.39%	1.13%	1.43%	0.00%	0.00%
ALU (Butterfly_Multiplication_Ur	8.13%	2.78%	10.41%	7.76%	1.13%	0.00%	0.00%	0.00%
A0 (Complex_Add)	1.10%	0.35%	1.46%	1.08%	0.06%	0.00%	0.00%	0.00%
Buffer5_0 (Buffer)	0.09%	0.07%	0.14%	0.01%	0.23%	0.00%	0.00%	0.00%
Buffer12_0 (Buffer__paramete	0.11%	0.06%	0.38%	0.05%	0.19%	0.00%	0.00%	0.00%
Buffer12_1 (Buffer__paramete	0.26%	0.10%	0.61%	0.20%	0.20%	0.00%	0.00%	0.00%
DivN (FP_Complex_Divide_N)	0.00%	0.06%	0.25%	0.00%	0.00%	0.00%	0.00%	0.00%
M0 (Complex_Mul)	5.06%	1.41%	6.64%	4.95%	0.34%	0.00%	0.00%	0.00%
S0 (Complex_Sub)	0.68%	0.32%	1.30%	0.66%	0.06%	0.00%	0.00%	0.00%
S1 (Complex_Sub_1)	0.87%	0.41%	1.35%	0.85%	0.06%	0.00%	0.00%	0.00%
Control_Unit (Control_FFTConv)	0.53%	0.06%	1.16%	0.53%	0.00%	0.00%	0.00%	0.00%
RAM_Dual_Port (RAM)	0.91%	0.00%	1.30%	0.91%	0.00%	1.43%	0.00%	0.00%
ROM (ROM_device)	0.05%	0.09%	0.38%	0.05%	0.00%	0.00%	0.00%	0.00%

Figure 4.1: Convolution Utilization Overview

- **Pipeline Overhead:** The utilization of Slice Registers (3%) is proportional to the pipeline depth (5-7 stages), confirming that the pipelining strategy did not incur excessive area penalties.

4.2 Timing Analysis and Performance

The system was constrained to a target clock period of 8.0ns (125MHz). The timing results in Table 4.2 confirm that all constraints were met.

Analysis:

- **Frequency Goal Met:** With a positive WNS of +0.265ns, the design operates reliably at 125MHz.
- **Critical Path:** The worst-case path is located in the *Complex Multiplier* stage (Delay:

Table 4.2: Timing Performance Summary (Target: 8.000 ns)

Parameter	**Value**	**Unit**
Worst Negative Slack (WNS)	**+0.265**	ns
Total Critical Path Delay	7.607	ns
- Logic Delay	4.157	ns
- Net (Routing) Delay	3.450	ns
Logic Levels	9	Levels
High Fanout	102	Nets

Name	Slack ^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net D
Path 1	0.265	9	4	94	dut/RAM_Dual_..._0/CLKBWRCLK	dut/ALU/M0/u_m...ge1_reg[27]/D	7.236	3.794	3
Path 2	0.291	9	4	94	dut/RAM_Dual_..._0/CLKBWRCLK	dut/ALU/M0/u_m...ge1_reg[27]/D	7.213	3.922	3
Path 3	0.305	9	4	94	dut/RAM_Dual_..._0/CLKBWRCLK	dut/ALU/M0/u_m...ge1_reg[25]/D	7.456	4.014	3
Path 4	0.331	9	4	94	dut/RAM_Dual_..._0/CLKBWRCLK	dut/ALU/M0/u_m...ge1_reg[25]/D	7.433	4.142	3
Path 5	0.346	9	4	102	dut/RAM_Dual_..._0/CLKBWRCLK	dut/ALU/M0/u_m...ge1_reg[27]/D	7.401	3.951	3
Path 6	0.398	8	4	94	dut/RAM_Dual_..._0/CLKBWRCLK	dut/ALU/M0/u_m...ge1_reg[27]/D	7.240	3.677	3
Path 7	0.399	9	4	102	dut/RAM_Dual_..._0/CLKBWRCLK	dut/ALU/M0/u_m...ge1_reg[25]/D	7.607	4.157	3
Path 8	0.400	9	4	94	dut/RAM_Dual_..._0/CLKBWRCLK	dut/ALU/M0/u_m...ge1_reg[26]/D	7.361	3.919	3
Path 9	0.416	9	4	94	dut/RAM_Dual_..._0/CLKBWRCLK	dut/ALU/M0/u_m...ge1_reg[24]/D	7.345	3.903	3
Path 10	0.426	9	4	94	dut/RAM_Dual_..._0/CLKBWRCLK	dut/ALU/M0/u_m...ge1_reg[26]/D	7.338	4.047	3

Figure 4.2: Convolution Timing Overview

7.607ns). The delay distribution is balanced between Logic (55%) and Routing (45%), indicating a healthy placement without severe congestion.

- **Pipeline Depth:** The critical path spans 9 logic levels. This confirms that the chosen pipeline depth was necessary; reducing the pipeline stages would likely have increased logic levels per cycle, violating the 8ns constraint.

4.3 Power Consumption Profile

The power analysis results, derived from the implemented netlist under typical process conditions (25°C ambient), are detailed in Table 4.3.

Analysis:

- **Low Power Profile:** The total consumption of 0.164W is minimal, making the IP Core suitable for power-constrained embedded applications.
- **Dominant Static Power:** Static power accounts for 63% of the total, which is inherent

Table 4.3: On-Chip Power Consumption Analysis

Power Component	**Value (W)**	**Percentage**
Total On-Chip Power	**0.164**	**100%**
Device Static Power	0.104	63%
Dynamic Power	0.060	37%
- Clocks	0.024	(40% of Dynamic)
- Signals	0.013	(22% of Dynamic)
- Logic	0.013	(21% of Dynamic)
- BRAM	0.010	(17% of Dynamic)

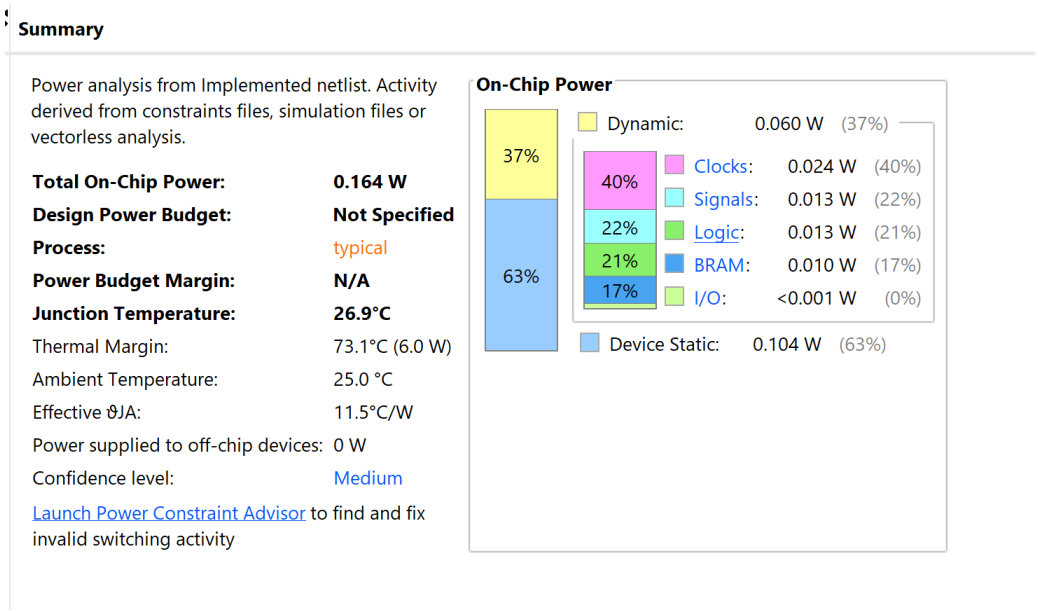

Figure 4.3: Convolution Power Consumption Overview

to the FPGA fabric. The Dynamic power is remarkably low (0.06W), reflecting efficient clock gating and localized switching activity within the pipeline.

- **Thermal Stability:** The junction temperature is estimated at 26.9°C, providing a safety margin of 73.1°C. No active cooling solution is required.

5 Simulation and Verification

To ensure the correctness of the hardware implementation, a rigorous verification process was conducted using a dual-environment approach: a standard **Verilog HDL** Testbench for hardware simulation and a Python script acting as the Golden Model for algorithmic validation.

5.1 Testbench Environment Setup

The verification environment is designed to simulate real-world data processing using standard Verilog I/O system tasks. The Testbench performs the following operations:

1. **Data Injection:** Reads input vectors from an external text file (`input_data.data`) using the `$readmemh` system task. The test dataset consists of **two independent random sequences**, each containing **128 Floating-Point samples** ($N = 128$). The data is stored in 32-bit hexadecimal format to match the hardware memory interface.
2. **DUT Stimulation:** Drives the control signals (`start`, `rst_n`) and feeds the random input sequences into the Convolution Core via the AXI-Stream interface protocol.
3. **Response Capture:** Monitors the output port and writes the resulting convolution values to a log file (`output_result.txt`) using `$fwrite` for post-processing analysis.

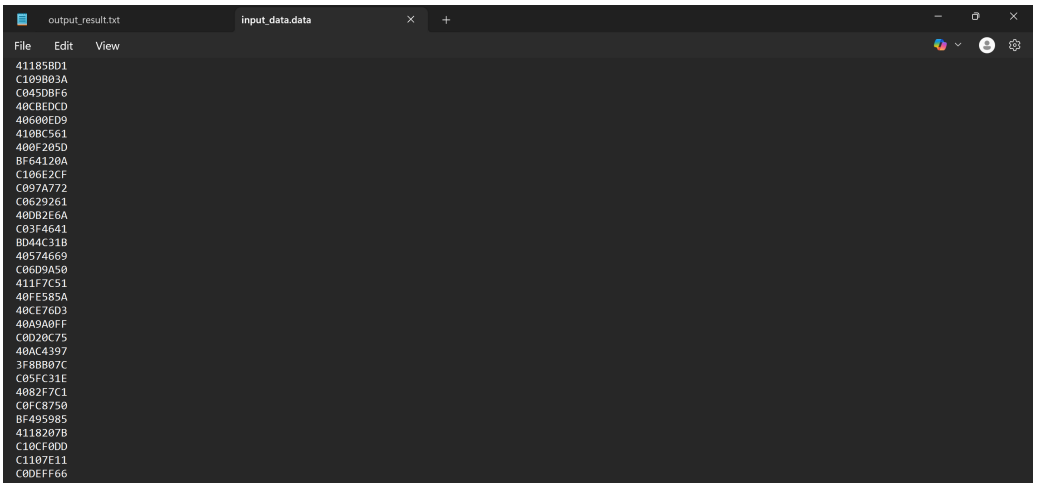

Figure 5.1: Input Data File containing 128-point Random FP Sequences (Hex Format)

5.2 Simulation Waveforms

The design was simulated using the Vivado Simulator with the 128-point datasets. Figure 5.2 illustrates the timing diagram of the processing core.

- The `start` signal triggers the Finite State Machine.
- The two 128-point sequences are loaded sequentially into the internal RAMs.
- The `done` signal is asserted high upon completion, indicating that the valid convolution results are available on the output bus.

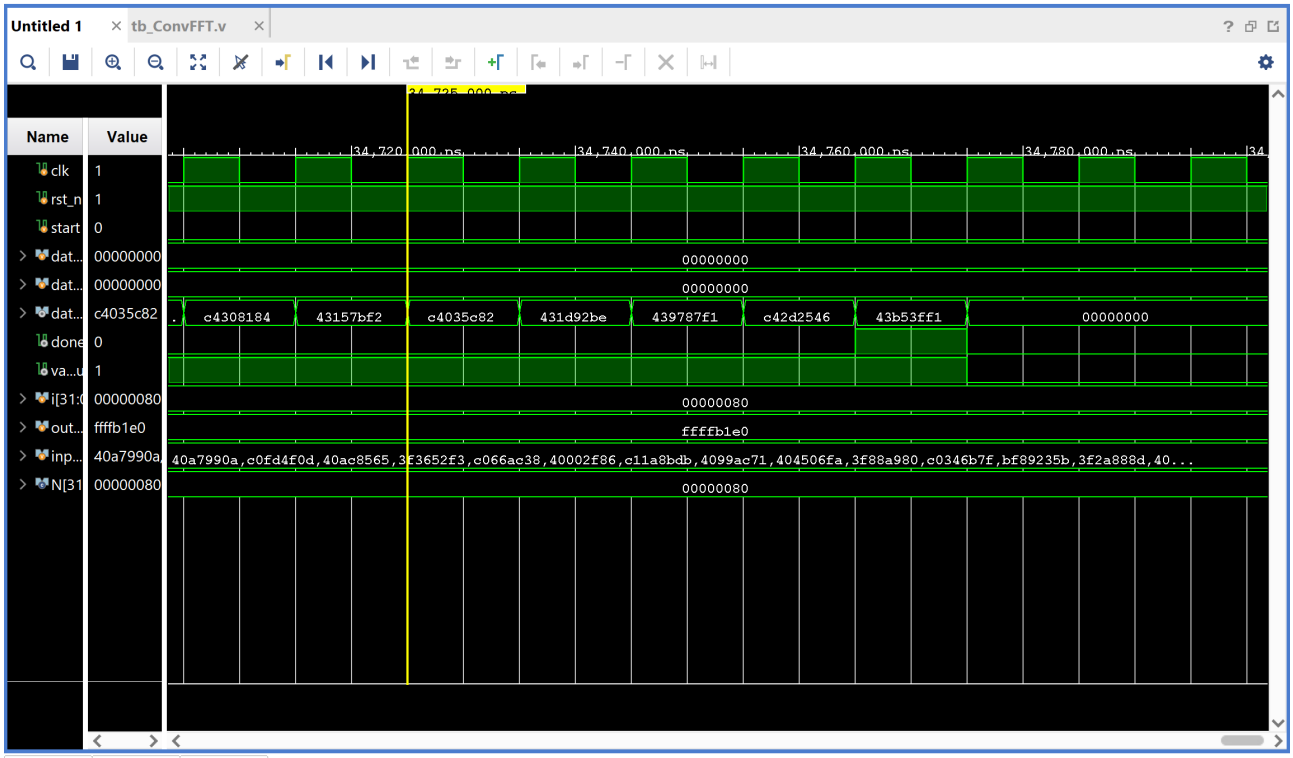

Figure 5.2: Simulation Waveform showing Input Loading and Output Generation

The raw output results captured from the simulation are stored in text format as shown below:

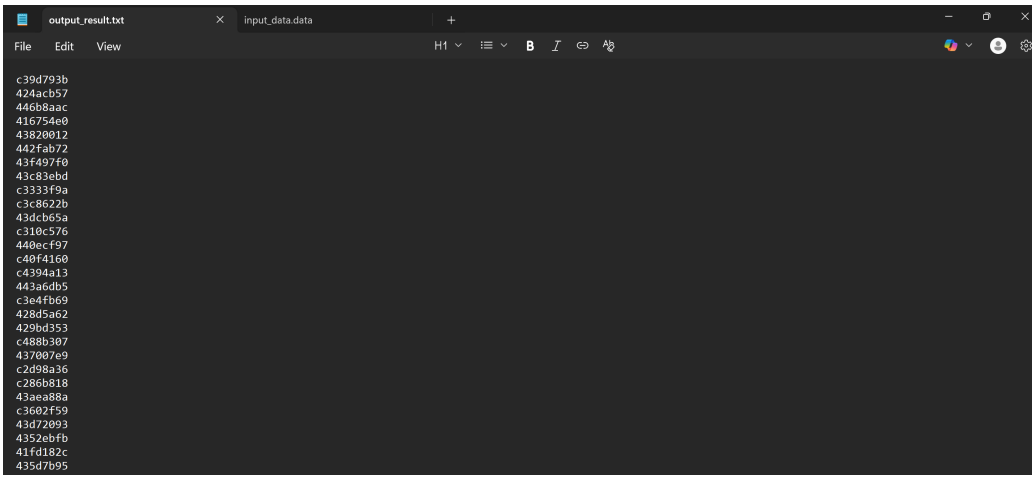

Figure 5.3: Output Result File generated by the Testbench

5.3 Cross-Verification with Python Golden Model

To validate the numerical accuracy of the hardware, a Python script was developed to calculate the reference convolution using standard scientific libraries (`numpy`). The script performs two key functions:

1. Generates the theoretical circular convolution result for the same two 128-point random input sequences.
2. Parses the hardware output file (`output_result.txt`), converts the IEEE 754 Hex values to Float, and compares them against the theoretical result.

Verification Result: The automated comparison confirms that the hardware output matches the expected logic for the $N = 128$ test case.

- **Status:** PASSED (Hardware output matches Circular Convolution).
- **Precision Error:** The maximum absolute error observed is **0.000872**.

This minor discrepancy is attributed to the inherent precision differences between the hardware's 32-bit Single-Precision Floating Point accumulation and Python's 64-bit Double-Precision calculations. The error falls within the acceptable range for single-precision DSP applications.

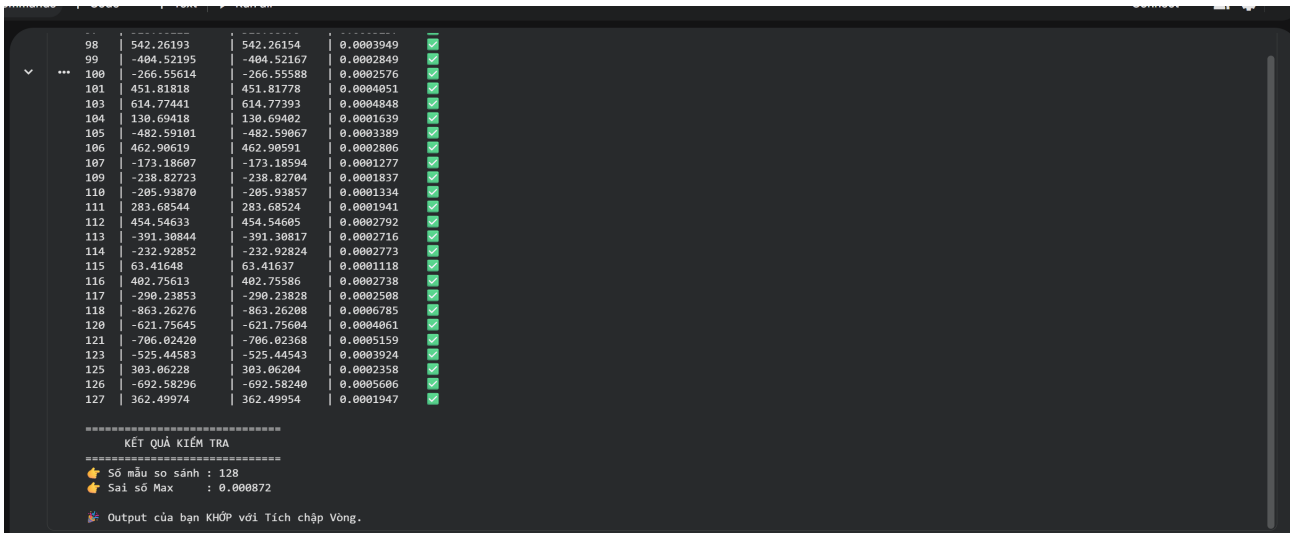

Figure 5.4: Python Verification Script Result: Validating Hardware Accuracy

References

[1] John G. Proakis and Dimitris G. Manolakis. *Digital Signal Processing: Principles, Algorithms, and Applications*. 4th. Prentice Hall, 2007. Chap. 8.